



TESTES UNITÁRIOS INVENTORY MASTERS

Documentação da suíte de testes unitários

Miguel Cássio Braga Duarte



SUMÁRIO

SUMÁRIO	2
1. Introdução	3
AcessoControllerTests	4
DashboardControllerTests	5
HomeControllerTests	6
KinectApiControllerTests	8
MedicoesControllerTests	10
NotificacoesControllerTests	11
ParametrosControllerTests	13
ParceirosControllerTests	15
PerfisControllerTests	18
UsuariosControllerTests	19



1. Introdução

Este documento apresenta a documentação dos testes unitários desenvolvidos para o Inventory Masters. A suíte foi construída com o objetivo de verificar os principais comportamentos das controllers, contemplando cenários de sucesso, validações de entrada, tratamento de exceções, respostas HTTP, persistência simulada e integrações com serviços como SignalR. Para manter os testes isolados da infraestrutura, foram utilizados mocks e objetos simulados das dependências.

AcessoControllerTests

A classe AcessoControllerTests implementa a validação das rotas HTTP GET de navegação e autenticação da controller de acesso, utilizando a alocação não inicializada de memória para isolar dependências de infraestrutura.

✓ AcessoControllerTests (3)	63 ms
✓ Login_DeveRetornarView	2 ms
✓ Negado_DeveRetornarView	< 1 ms
✓ ValidarToken_DeveRetornarView	61 ms
✓ DashboardControllerTests (2)	129 ms

Estratégia de Inicialização do Construtor

- **FormatterServices.GetUninitializedObject:** Utilizado para criar instâncias de TokenAcessoKinectService, EmailTokenService, UsuariosRepository e LogsSistemaRepository ignorando a execução de seus construtores padrão, impedindo conexões com banco de dados ou provedores de e-mail.
- **ConfigurationBuilder e LoggerFactory:** Instanciam objetos reais leves de configuração e logs para satisfazer a injeção de dependências do AcessoController.
- **DefaultHttpContext:** Configurado no ControllerContext para simular o contexto HTTP do ASP.NET Core, prevenindo exceções de referência nula ao ler cookies, cabeçalhos ou sessões.

Casos de Teste Mapeados

- **Login_DeveRetornarView**
 - **Ação Avaliada:** _controller.Login()
 - **Tipo de Retorno:** ViewResult
 - **Modelo Esperado:** LoginEmailViewModel
 - **Propósito:** Garante que a tela de login inicial é renderizada com o formulário de e-mail zerado.
- **ValidarToken_DeveRetornarView**
 - **Ação Avaliada:** _controller.ValidarToken()
 - **Tipo de Retorno:** ViewResult
 - **Modelo Esperado:** ValidarTokenViewModel
 - **Propósito:** Garante que a tela de validação do código de acesso é exibida com o modelo correto.
- **Negado_DeveRetornarView**
 - **Ação Avaliada:** _controller.Negado()
 - **Tipo de Retorno:** ViewResult
 - **Modelo Esperado:** N/A
 - **Propósito:** Valida o direcionamento para a View de erro de acesso negado (HTTP 403).

Detalhamento Técnico e Justificativas do Código

- **Login_DeveRetornarView**

```
Assert.IsType<ViewResult>(resultado);  
Assert.NotNull(view.Model);  
Assert.IsType<LoginEmailViewModel>(view.Model);
```

- **Justificativa:** A rota GET de login precisa entregar um modelo inicial para a View compilar o formulário sem apontar erro de referência nula no Razor (@model LoginEmailViewModel).
- **ValidarToken_DeveRetornarView**

```
Assert.IsType<ViewResult>(resultado);  
Assert.NotNull(view.Model);  
Assert.IsType<ValidarTokenViewModel>(view.Model);
```

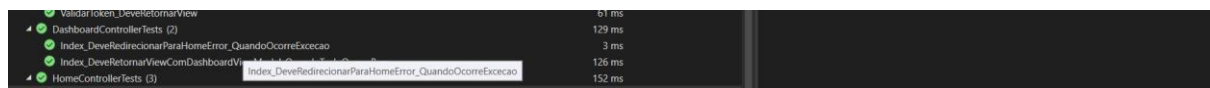
- **Justificativa:** Assegura que o fluxo da etapa de validação em dois fatores (2FA) constrói a estrutura necessária antes do usuário digitar o código enviado por e-mail.
- **Negado_DeveRetornarView**

```
Assert.IsType<ViewResult>(resultado);
```

- **Justificativa:** A rota de acesso negado serve apenas como página informativa estática, dispensando a necessidade de validação de modelos anexados.

DashboardControllerTests

A classe DashboardControllerTests valida a agregação de indicadores e métricas operacionais do painel principal, utilizando a biblioteca Moq para isolar as 5 interfaces de repositório e o serviço de logging do ASP.NET Core.



ValidarToken_DeveRetornarView	61 ms
DashboardControllerTests (2)	129 ms
Index_DeveRedirecionarParaHomeError_QuandoOcorreExcecao	3 ms
Index_DeveRetornarViewComDashboardViewModel_QuandoOcorreExcecao	126 ms
HomeControllerTests (3)	132 ms

Estratégia de Inicialização do Construtor

- **Mock<T> por Interfaces:** Emprega dublês para IMedicaoVolumeRepository, INotificacaoRepository, IParceirosRepository, IParametrosSistemaRepository, IUsuariosRepository e ILogger<DashboardController>.
- **Injeção de Dependência:** Injeta as propriedades .Object de cada mock diretamente na instância da DashboardController, garantindo isolamento total de acessos ao banco de dados SQL.

Casos de Teste Mapeados

- **Index_DeveRetornarViewComDashboardViewModel_QuandoTudoOcorreBem**
 - **Ação Avaliada:** await _controller.Index()
 - **Tipo de Retorno:** ViewResult
 - **Modelo/Resultado Esperado:** DashboardViewModel (PercentualOcupacao = 50)

- **Propósito:** Valida a compilação das 5 fontes de dados e o cálculo correto do indicador de ocupação.
- **Index_DeveRedirecionarParaHomeError_QuandoOcorreExcecao**
 - **Ação Avaliada:** await _controller.Index()
 - **Tipo de Retorno:** RedirectToActionResult
 - **Modelo/Resultado Esperado:** Action = "Error", Controller = "Home"
 - **Propósito:** Assegura a captura defensiva de exceções e o redirecionamento amigável em falhas.

Detalhamento Técnico e Justificativas do Código

- **Index_DeveRetornarViewComDashboardViewModel_QuandoTudoOcorreBem**

```
_medicaoRepoMock.Setup(repo => repo.ListarPorEmpresa(It.IsAny<string>()))  
.ReturnsAsync(new List<MedicaoVolume> { new MedicaoVolume { VolumeMedido = 5000 } });
```

```
_parametrosRepoMock.Setup(repo => repo.Buscar())  
.Returns(new ParametrosSistema { CapacidadeMaxima = 10000 });
```

```
Assert.Equal(50, model.PercentualOcupacao);
```

- **Justificativa:** A rota Index compila dados de múltiplos módulos. O teste simula uma medição de 5.000 m³ e capacidade total de 10.000 m³, garantindo que a controller executa a regra de negócio ($\frac{5000}{10000} \times 100 = 50\%$) antes de entregar o modelo preenchido.

- **Index_DeveRedirecionarParaHomeError_QuandoOcorreExcecao**

```
_parceirosRepoMock.Setup(repo => repo.ListarPorEmpresa(It.IsAny<string>()))  
.ThrowsAsync(new Exception("Erro simulado no banco"));
```

```
var redirectResult = Assert.IsType<RedirectToActionResult>(resultado);  
Assert.Equal("Error", redirectResult.ActionName);  
Assert.Equal("Home", redirectResult.ControllerName);
```

- **Justificativa:** Garante a resiliência do painel. Caso qualquer um dos repositórios falhe por queda de conexão ou timeout, o bloco try-catch interno da controller intercepta o erro sem quebrar o servidor, redirecionando o fluxo para /Home/Error.

HomeControllerTests

A classe HomeControllerTests valida o redirecionamento da raiz da aplicação para o painel principal (DashboardController) e o pipeline global de tratamento e diagnóstico de exceções do ASP.NET Core (ExceptionHandlerPathFeature).

HomeControllerTests (3)	
Error_DeveRetornarViewComModel_ERegistrarLog_QuandoHouverExcecao	Index_DeveRetornarViewComDashboardViewModel_QuandoTudoCorrerEm
Error_DeveRetornarViewComModel_SemRegistrarLog_QuandoNaoHouverExcecao	1 ms
Index_DeveRedirecionarParaDashboard	< 1 ms
KinectApiControllerTests (4)	149 ms

Estratégia de Simulação do HttpContext e Diagnostics

- **IExceptionHandlerPathFeature:** Mockado para simular a captura de exceções não tratadas disparadas em qualquer ponto da aplicação e interceptadas pelo middleware de erro.
- **IFeatureCollection e RequestServices:** Configurados no HttpContext simulado para fornecer as features de diagnóstico de exceção e o resolvidor de dependências (IServiceProvider).
- **ITempDataDictionary e TraceIdentifier:** Injetados para simular o contexto completo da requisição HTTP e evitar falhas de renderização da View de erro.

Casos de Teste Mapeados

- **Index_DeveRedirecionarParaDashboard**
 - **Ação Avaliada:** controller.Index()
 - **Tipo de Retorno:** RedirectToActionResult
 - **Modelo/Resultado Esperado:** Action = "Index", Controller = "Dashboard"
 - **Propósito:** Garante que a rota raiz da aplicação redireciona automaticamente para o dashboard.
- **Error_DeveRetornarViewComModel_ERegistrarLog_QuandoHouverExcecao**
 - **Ação Avaliada:** controller.Error()
 - **Tipo de Retorno:** ViewResult
 - **Modelo/Resultado Esperado:** ErrorViewModel (RequestId preenchido)
 - **Propósito:** Valida a extração da exceção do middleware, geração do ID de rastreamento e disparo do log (LogLevel.Error).
- **Error_DeveRetornarViewComModel_SemRegistrarLog_QuandoNaoHouverExcecao**
 - **Ação Avaliada:** controller.Error()
 - **Tipo de Retorno:** ViewResult
 - **Modelo/Resultado Esperado:** ErrorViewModel (RequestId = "trace-id-456")
 - **Propósito:** Garante o comportamento gracioso quando a página de erro é acessada diretamente (sem exceção registrada).

Detalhamento Técnico e Justificativas do Código

- **Index_DeveRedirecionarParaDashboard**

```
var redirectResult = Assert.IsType<RedirectToActionResult>(resultado);  
Assert.Equal("Index", redirectResult.ActionName);  
Assert.Equal("Dashboard", redirectResult.ControllerName);
```

- **Justificativa:** A HomeController atua como porta de entrada. O teste confirma que acessar a página inicial navega o usuário imediatamente para o módulo central do sistema (Dashboard/Index).

- **Error_DeveRetornarViewComModel_ERegistrarLog_QuandoHouverExcecao**

```
exceptionFeatureMock.Setup(f => f.Error).Returns(new Exception("Erro de teste simulado"));
```

```
loggerMock.Verify(
    x => x.Log(
        It.Is<LogLevel>(l => l == LogLevel.Error),
        It.IsAny<EventId>(),
        It.Is<It.IsAnyType>((v, t) => true),
        It.IsAny<Exception>(),
        It.Is<Func<It.IsAnyType, Exception, string>>((v, t) => true)),
    Times.Once);
```

- **Justificativa:** O teste simula a exceção no `IExceptionHandlerPathFeature` para garantir que a `HomeController` resgate os detalhes da exceção, registre o log crítico uma única vez (`Times.Once`) e exiba a View amigável com o código de rastreamento (`RequestId`).

- **Error_DeveRetornarViewComModel_SemRegistrarLog_QuandoNaoHouverExcecao**

```
featuresMock.Setup(f =>
    f.Get<IExceptionHandlerPathFeature>()).Returns((IExceptionHandlerPathFeature?)null);
```

```
Assert.Equal("trace-id-456", model.RequestId);
```

- **Justificativa:** Se um usuário digitar `/Home/Error` diretamente no navegador, não haverá exceção armazenada nas features do pipeline. O teste confirma que o sistema lida com esse cenário sem lançar `NullReferenceException`, utilizando o `TraceIdentifier` nativo.

KinectApiControllerTests

A classe `KinectApiControllerTests` valida os endpoints REST de autenticação do hardware Kinect, cobrindo solicitação, envio por e-mail, auditoria e validação de tokens de acesso.

KinectApiControllerTests (4)	149 ms
SolicitarToken_DeveRetornarBadRequest_QuandoModelStateInvalido	< 1 ms
SolicitarToken_DeveRetornarOkEnviarEmail_QuandoSucesso	145 ms
ValidarToken_DeveRetornarOk_QuandoTokenValido	1 ms
ValidarToken_DeveRetornarUnauthorized_QuandoTokenInvalido	3 ms

Estratégia de Configuração e Mocks

- **IConfigurationSection Mockado:** Configura a chave `KinectAccess:TokenValidityMinutes` para retornar "15", garantindo que a controller leia o tempo de expiração do token sem depender de arquivos `appsettings.json` externos.
- **Isolamento de Efeitos Colaterais:** Mocka as interfaces `ITokenAcessoKinectService`, `IEmailTokenService` e `ILogSistemaRepository` para permitir a verificação do envio de e-mails e gravação de logs sem serviços externos.

Casos de Teste Mapeados

- **SolicitarToken_DeveRetornarBadRequest_QuandoModelStateInvalido**
 - **Ação Avaliada:** `await _controller.SolicitarToken(request)`
 - **Status HTTP/Retorno:** 400 Bad Request (`BadRequestObjectResult`)

- **Propósito:** Intercepta payloads com dados ausentes ou inválidos antes de acionar regras de negócio.
- **SolicitarToken_DeveRetornarOkEEEnviarEmail_QuandoSucesso**
 - **Ação Avaliada:** `await _controller.SolicitarToken(request)`
 - **Status HTTP/Retorno:** 200 OK (`OkObjectResult`)
 - **Propósito:** Confirma a emissão do token, envio do e-mail e registro no log de auditoria.
- **ValidarToken_DeveRetornarOk_QuandoTokenValido**
 - **Ação Avaliada:** `await _controller.ValidarToken(request)`
 - **Status HTTP/Retorno:** 200 OK (`OkObjectResult`)
 - **Propósito:** Libera o acesso quando o código informado pelo dispositivo Kinect é válido.
- **ValidarToken_DeveRetornarUnauthorized_QuandoTokenInvalido**
 - **Ação Avaliada:** `await _controller.ValidarToken(request)`
 - **Status HTTP/Retorno:** 401 Unauthorized (`UnauthorizedObjectResult`)
 - **Propósito:** Bloqueia a requisição quando o token informado está expirado ou é inexistente.

Detalhamento Técnico e Justificativas do Código

- **SolicitarToken_DeveRetornarBadRequest_QuandoModelStateInvalido**

`_controller.ModelState.AddModelError("Email", "E-mail é obrigatório");`

- **Justificativa:** Em testes unitários, os atributos de validação como `[Required]` não acionam o pipeline do ASP.NET Core automaticamente. Injetar o erro via `ModelState.AddModelError` garante a execução do bloco de verificação defensiva `if (!ModelState.IsValid)`.

- **SolicitarToken_DeveRetornarOkEEEnviarEmail_QuandoSucesso**

`_tokenServiceMock.Setup(s => s.GerarTokenParaEmail(request.Email))
.ReturnsAsync((true, "Token gerado com sucesso.", "123456", usuarioFake));`

`_emailServiceMock.Verify(s => s.EnviarTokenKinect(It.IsAny<string>(), It.IsAny<string>(),
It.IsAny<string>(), It.IsAny<int>()), Times.Once);`

`_logsRepositoryMock.Verify(r => r.Registrar(It.IsAny<string>(), It.IsAny<string>(),
It.IsAny<string>(), It.IsAny<string>(), It.IsAny<string>(), It.IsAny<string>()), Times.Once);`

- **Justificativa:** O teste simula o retorno da tupla (`bool, string, string, Usuario`) do serviço e utiliza `.Verify(..., Times.Once)` para assegurar que o envio do e-mail com o código de 6 dígitos e o registro do log de segurança são executados uma vez.

- **ValidarToken_DeveRetornarUnauthorized_QuandoTokenInvalido**

`Assert.IsType<UnauthorizedObjectResult>(resultado);`

- **Justificativa:** Para endpoints de API, credenciais ou tokens inválidos devem retornar o status nativo HTTP 401 Unauthorized, assegurando aderência aos padrões do protocolo HTTP.

MedicoesControllerTests

A classe MedicoesControllerTests valida a listagem paginada, o cálculo estatístico de métricas de volume e os endpoints de resumo em JSON da controller de medições.

MedicoesControllerTests (4)	204 ms	MVC_Inventory_Master.Tests
Index_DeveRetornarViewComListaPaginadaEViewBagPreenchida	78 ms	Testes em grupo: 60
Index_QuandoOcorrerExcecao_DeveRedirecionarParaError	1 ms	Duração total: 1,6 s
Summary_DeveRetornarJsonComResumo	< 1 ms	Resultados
Summary_QuandoOcorrerExcecao_DeveRetornarStatusCode500	125 ms	60 Aprovado

Estratégia de Configuração e Mocks

- **IHubContext<MedicaoHub> Mockado:** Simula a infraestrutura de WebSockets do SignalR, permitindo validar a controller sem manter conexões abertas com clientes.
- **Validação de Agregações na ViewBag:** Assegura que a controller calcula métricas dinâmicas diretamente na View (totais por status e volume médio) a partir dos dados do repositório.

Casos de Teste Mapeados

- **Index_DeveRetornarViewComListaPaginadaEViewBagPreenchida**
 - **Ação Avaliada:** await _controller.Index(1, null, null, null, null)
 - **Status/Retorno:** ViewResult com List<MedicaoVolume> e ViewBag populada
 - **Propósito:** Valida a renderização da tabela e o cálculo das métricas dinâmicas (TotalRegistros, TotalNormal, TotalAlerta, VolumeMedio).
- **Index_QuandoOcorrerExcecao_DeveRedirecionarParaError**
 - **Ação Avaliada:** await _controller.Index()
 - **Status/Retorno:** RedirectToActionResult (Home/Error)
 - **Propósito:** Assegura que falhas de conexão com o banco na busca de medições não quebram a aplicação.
- **Summary_DeveRetornarJsonComResumo**
 - **Ação Avaliada:** await _controller.Summary()
 - **Status/Retorno:** JsonResult com MedicaoSummary
 - **Propósito:** Confirma o fornecimento de dados consolidados em formato JSON para os gráficos da interface.
- **Summary_QuandoOcorrerExcecao_DeveRetornarStatusCode500**
 - **Ação Avaliada:** await _controller.Summary()
 - **Status/Retorno:** ObjectResult com StatusCode = 500
 - **Propósito:** Assegura a emissão do status HTTP 500 Internal Server Error em falhas na geração do resumo.

Detalhamento Técnico e Justificativas do Código

- **Index_DeveRetornarViewComListaPaginadaEViewBagPreenchida**

```
var medicoesExemplo = new List<MedicaoVolume>
{
    new() { DataHora = DateTime.UtcNow, VolumeMedido = 100, Status = "Normal" },

```

```
new() { DataHora = DateTime.UtcNow.AddMinutes(-5), VolumeMedido = 250, Status = "Alerta"
}
};
```

```
Assert.Equal(2, _controller.ViewBag.TotalRegistros);
Assert.Equal(1, _controller.ViewBag.TotalNormal);
Assert.Equal(1, _controller.ViewBag.TotalAlerta);
Assert.Equal(175, _controller.ViewBag.VolumeMedio);
```

- **Justificativa:** A controller processa os dados brutos do repositório para extrair contadores e médias. O teste simula os registros e valida se as operações aritméticas da ViewBag correspondem exatamente à média e contagem esperadas ($\frac{100+250}{2} = 175$).

- **Summary_DeveRetornarJsonComResumo**

```
var jsonResult = Assert.IsType<JsonResult>(result);
Assert.Equal(resumoEsperado, jsonResult.Value);
```

- **Justificativa:** A rota Summary é consumida via requisições AJAX do front-end. O teste valida se o modelo retornado é do tipo JsonResult e transporta o objeto MedicaoSummary sem alterações.

- **Summary_QuandoOcorrerExcecao_DeveRetornarStatusCode500**

```
var statusResult = Assert.IsType<ObjectResult>(result);
Assert.Equal(500, statusResult.StatusCode);
```

- **Justificativa:** Diferente de rotas de navegação visual, rotas de API/AJAX devem sinalizar falhas enviando o status HTTP 500 para que o cliente JavaScript identifique o erro no bloco .catch().

NotificacoesControllerTests

A classe NotificacoesControllerTests valida o gerenciamento e filtragem de notificações, processamento de confirmações de coleta e emissão de alertas em tempo real via SignalR (NotificacaoHub).

NotificacoesControllerTests (6)	198 ms
AceitarColeta_ComIdNuloOuVazio_DeveRetornarBadRequest	1 ms
AceitarColeta_ComSucesso_DeveAtualizarStatusENotificarSignalR	10 ms
AceitarColeta_QuandoFalharAtualizacao_DeveRetornar500	1 ms
AceitarColeta_QuandoOcorrerExcecao_DeveRetornar500	129 ms
Index_DeveRetornarViewComListaPaginadaEViewBagPreenchida	56 ms
Index_QuandoOcorrerExcecao_DeveRedirecionarParaError	1 ms
ParametrosControllerTests (11)	134 ms

Estratégia de Mocking do SignalR e Repositórios

- **Hierarquia do SignalR:** Configura o encadeamento de chamadas do hub interceptando IHubContext.Clients.All, direcionando a execução para a interface IClientProxy.
- **Isolamento de Dados:** Simula INotificacaoRepository e IParceirosRepository para testar a agregação das métricas do painel e o processamento de alterações de status no banco SQL.

Casos de Teste Mapeados

- **Index_DeveRetornarViewComListaPaginadaEViewBagPreenchida**
 - **Ação Avaliada:** `await _controller.Index(1, ...)`
 - **Status/Retorno:** `ViewResult` com `List<Notificacao>` e `ViewBag` preenchida
 - **Propósito:** Confirma a listagem e os contadores de métricas por status (`TotalSucesso`, `TotalErro`, `TotalPendente`).
- **Index_QuandoOcorrerExcecao_DeveRedirecionarParaError**
 - **Ação Avaliada:** `await _controller.Index()`
 - **Status/Retorno:** `RedirectToActionResult` (`Home/Error`)
 - **Propósito:** Assegura que erros de infraestrutura na busca de notificações redirecionam o usuário em segurança.
- **AceitarColeta_ComIdNuloOuVazio_DeveRetornarBadRequest**
 - **Ação Avaliada:** `await _controller.AceitarColeta("")`
 - **Status/Retorno:** `400 Bad Request` (`BadRequestObjectResult`)
 - **Propósito:** Bloqueia requisições sem o parâmetro obrigatório de identificação.
- **AceitarColeta_ComSucesso_DeveAtualizarStatusENotificarSignalR**
 - **Ação Avaliada:** `await _controller.AceitarColeta(id)`
 - **Status/Retorno:** `200 OK` (`OkObjectResult`)
 - **Propósito:** Valida a transição para o status "Aceito" e a transmissão da mensagem em tempo real para o SignalR.
- **AceitarColeta_QuandoFalharAtualizacao_DeveRetornar500**
 - **Ação Avaliada:** `await _controller.AceitarColeta(id)`
 - **Status/Retorno:** `500 Internal Server Error` (`ObjectResult`)
 - **Propósito:** Intercepta falhas de gravação no banco quando o repositório retorna false.
- **AceitarColeta_QuandoOcorrerExcecao_DeveRetornar500**
 - **Ação Avaliada:** `await _controller.AceitarColeta(id)`
 - **Status/Retorno:** `500 Internal Server Error` (`ObjectResult`)
 - **Propósito:** Captura exceções não tratadas durante o fluxo de confirmação e responde com status HTTP 500.

Detalhamento Técnico e Justificativas do Código

- **Index_DeveRetornarViewComListaPaginadaEViewBagPreenchida**

```
Assert.Equal(3, _controller.ViewBag.TotalRegistros);  
Assert.Equal(1, _controller.ViewBag.TotalSucesso);  
Assert.Equal(1, _controller.ViewBag.TotalErro);  
Assert.Equal(1, _controller.ViewBag.TotalPendente);
```

- **Justificativa:** A controller sintetiza métricas para renderizar cartões na interface. O teste cria um cenário heterogêneo para garantir que a soma dos contadores na ViewBag corresponde exatamente ao agrupamento do repositório.

- **AceitarColeta_ComSucesso_DeveAtualizarStatusENotificarSignalR**

```
_mockRepo.Verify(r => r.AtualizarStatus(notificacaoId, "Aceito"), Times.Once);
```

```
_mockClientProxy.Verify(
```

```
    p => p.SendCoreAsync("ReceberNotificacao", It.Is<object[]>(o => o.Length > 0 &&  
o[0].ToString() == "Uma nova coleta foi aceita!"), default),
```

```
    Times.Once);
```

- **Justificativa:** Ao aceitar uma coleta, o sistema precisa persistir o novo status e notificar os usuários no painel via WebSocket. O teste utiliza .Verify no IClientProxy para assegurar que a chamada SendCoreAsync foi executada com a mensagem esperada.

ParametrosControllerTests

A classe ParametrosControllerTests valida a gestão das configurações operacionais do sistema (limites de capacidade volumétrica, percentuais de alerta, calibração do sensor Kinect e restauração de padrões fabris).

ParametrosControllerTests (11)	134 ms
Index_DeveRetornarViewComParametros	2 ms
Index_QuandoOcorrerExcecao_DeveRetornarViewComModeloVazioETempDataErro	1 ms
IniciarCalibracao_ComSucesso_DeveAtivarCalibracaoESalvar	1 ms
IniciarCalibracao_QuandoOcorrerExcecao_DeveDefinirTempDataErroRedirecionar	< 1 ms
RestaurarPadroes_ComSucesso_DeveObterPadroesESalvar	1 ms
RestaurarPadroes_QuandoOcorrerExcecao_DeveDefinirTempDataErroRedirecionar	< 1 ms
Salvar_ComAlteracoes_DeveSalvarDefinirTempDataSucesso	1 ms
Salvar_ComCapacidadeMinimaMaiorOuIgualAMaxima_DeveAdicionarErroNoModelStateRetornarViewIndex	< 1 ms
Salvar_ComModelStateInvalido_DeveRetornarViewIndexComModelo	< 1 ms
Salvar_QuandoOcorrerExcecao_DeveDefinirTempDataErroRetornarViewIndex	1 ms
Salvar_SemAlteracoes_DeveDefinirTempDataAvisoRedirecionarParaIndex	127 ms

Estratégia de Configuração e Mocks

- **Instanciação Manual de TempData:** Injeta um TempDataDictionary acoplado a um DefaultHttpContext para testar mensagens do tipo flash (TempData["Sucesso"], TempData["Erro"] e TempData["Aviso"]).
- **Idempotência:** Mapeia a comparação entre o modelo enviado pela interface e o estado atual retornado pelo repositório (Buscar()), impedindo chamadas desnecessárias de gravação.

Casos de Teste Mapeados

- **Index_DeveRetornarViewComParametros**
 - **Ação Avaliada:** _controller.Index()
 - **Status/Retorno:** ViewResult com ParametrosSistema
 - **Propósito:** Valida o carregamento dos limites atuais na interface.
- **Index_QuandoOcorrerExcecao_DeveRetornarViewComModeloVazioETempDataErro**
 - **Ação Avaliada:** _controller.Index()
 - **Status/Retorno:** ViewResult + TempData["Erro"]
 - **Propósito:** Assegura a exibição defensiva da tela com alerta de erro em falhas de leitura no banco.
- **Salvar_ComModelStateInvalido_DeveRetornarViewIndexComModelo**
 - **Ação Avaliada:** _controller.Salvar(model)
 - **Status/Retorno:** ViewResult("Index")

- **Propósito:** Impede o processamento quando campos obrigatórios não são preenchidos.
- **Salvar_ComCapacidadeMinimaMaiorOuIgualAMaxima_DeveAdicionarErroNoModelState**
 - **Ação Avaliada:** `_controller.Salvar(model)`
 - **Status/Retorno:** `ViewResult("Index") + Erro no ModelState`
 - **Propósito:** Executa a validação de regra de negócio (*CapacidadeMinima* $\geq$ *CapacidadeMaxima*).
- **Salvar_SemAlteracoes_DeveDefinirTempDataAvisoERedirecionarParaIndex**
 - **Ação Avaliada:** `_controller.Salvar(model)`
 - **Status/Retorno:** `RedirectToActionResult("Index") + TempData["Aviso"]`
 - **Propósito:** Evita escritas desnecessárias no banco quando nenhum parâmetro é modificado.
- **Salvar_ComAlteracoes_DeveSalvarEDefinirTempDataSucesso**
 - **Ação Avaliada:** `_controller.Salvar(model)`
 - **Status/Retorno:** `RedirectToActionResult("Index") + TempData["Sucesso"]`
 - **Propósito:** Valida a atualização bem-sucedida das novas configurações no repositório.
- **Salvar_QuandoOcorrerExcecao_DeveDefinirTempDataErroERetornarViewIndex**
 - **Ação Avaliada:** `_controller.Salvar(model)`
 - **Status/Retorno:** `ViewResult("Index") + TempData["Erro"]`
 - **Propósito:** Trata exceções lançadas durante a persistência dos parâmetros.
- **IniciarCalibracao_ComSucesso_DeveAtivarCalibracaoESalvar**
 - **Ação Avaliada:** `_controller.IniciarCalibracao()`
 - **Status/Retorno:** `RedirectToActionResult("Index") + TempData["Sucesso"]`
 - **Propósito:** Confirma o acionamento da bandeira de calibração do sensor volumétrico.
- **IniciarCalibracao_QuandoOcorrerExcecao_DeveDefinirTempDataErroERedirecionar**
 - **Ação Avaliada:** `_controller.IniciarCalibracao()`
 - **Status/Retorno:** `RedirectToActionResult("Index") + TempData["Erro"]`
 - **Propósito:** Garante tratamento de erros na inicialização da calibração.
- **RestaurarPadroes_ComSucesso_DeveObterPadroesESalvar**
 - **Ação Avaliada:** `_controller.RestaurarPadroes()`
 - **Status/Retorno:** `RedirectToActionResult("Index") + TempData["Sucesso"]`
 - **Propósito:** Assegura que as configurações fabris padrão são resgatadas e gravadas.
- **RestaurarPadroes_QuandoOcorrerExcecao_DeveDefinirTempDataErroERedirecionar**
 - **Ação Avaliada:** `_controller.RestaurarPadroes()`
 - **Status/Retorno:** `RedirectToActionResult("Index") + TempData["Erro"]`
 - **Propósito:** Trata falhas durante a redefinição de fábrica do sistema.

Detalhamento Técnico e Justificativas do Código

- **Salvar_ComCapacidadeMinimaMaiorOuIgualAMaxima_DeveAdicionarErroNoModelStateERetornarViewIndex**

```
var model = new ParametrosSistema { CapacidadeMinima = 100, CapacidadeMaxima = 50 };
```

```
var result = _controller.Salvar(model);
```

```
Assert.False(_controller.ModelState.IsValid);
```

```
Assert.True(_controller.ModelState.ContainsKey(nameof(ParametrosSistema.CapacidadeMinima)));
```

- **Justificativa:** É uma regra do sistema que a capacidade mínima seja estritamente menor que a máxima. O teste valida se a controller insere a chave de erro diretamente na propriedade CapacidadeMinima do ModelState.

- **Salvar_SemAlteracoes_DeveDefinirTempDataAvisoERedirecionarParaIndex**

```
_mockRepo.Setup(r => r.Buscar()).Returns(parametrosAtuais);
```

```
var result = _controller.Salvar(modelSemAlteracao);
```

```
Assert.Equal("Nenhuma alteração foi realizada.", _controller.TempData["Aviso"]);
```

```
_mockRepo.Verify(r => r.Salvar(It.IsAny<ParametrosSistema>()), Times.Never);
```

- **Justificativa:** Clicar em "Salvar" sem alterar dados não deve disparar comandos no banco. O teste garante que a controller compara o modelo com o estado atual e impede a execução de Salvar (Times.Never).

- **IniciarCalibracao_ComSucesso_DeveAtivarCalibracaoESalvar**

```
var parametros = new ParametrosSistema { AtivarSistemaCalibracao = false };
```

```
_mockRepo.Setup(r => r.Buscar()).Returns(parametros);
```

```
var result = _controller.IniciarCalibracao();
```

```
Assert.True(parametros.AtivarSistemaCalibracao);
```

```
_mockRepo.Verify(r => r.Salvar(parametros), Times.Once);
```

- **Justificativa:** A rotina de calibração altera a propriedade AtivarSistemaCalibracao para true no banco para que o serviço em background inicie o procedimento. O teste valida a alteração desse estado booleano e sua persistência.

ParceirosControllerTests

A classe ParceirosControllerTests valida o ciclo de vida completo do cadastro de parceiros (CRUD), incluindo filtragem, paginação, exclusão e alternância de status ativo/inativo.

ParceirosControllerTests (15)	212 ms
AlternarStatus_AtivoFalse_DeveAtivarEInformarSucesso	< 1 ms
AlternarStatus_AtivoTrue_DeveDesativarEInformarSucesso	1 ms
Create_Get_DeveRetornarViewComNovoParceiroAtivo	< 1 ms
Create_Post_ComModelStateInvalido_DeveRetornarViewComModelo	< 1 ms
Create_Post_ComSucesso_DeveAdicionarERedirecionar	1 ms
Create_Post_QuandoOcorreExcecao_DeveRetornarViewTempDataErro	2 ms
DeleteConfirmed_ComSucesso_DeveExcluirERedirecionar	1 ms
Details_ComIdValido_DeveRetornarViewComParceiro	< 1 ms
Details_ComIdVazio_DeveRetornarBadRequest	< 1 ms
Details_QuandoNaoEncontrado_DeveRetornarNotFound	119 ms
Details_QuandoOcorreExcecao_DeveRedirecionarParaErro	< 1 ms
Edit_Post_ComAlteracoes_DeveAtualizarERedirecionar	3 ms
Edit_Post_SemAlteracoes_DeveDefinirViewBagAvisoERetornarView	5 ms
Index_DeveRetornarViewComListaPaginadaEViewBagPreenchida	79 ms
Index_QuandoOcorreExcecao_DeveRedirecionarParaErro	1 ms

Estratégia de Mocking e Infraestrutura

- **Isolamento de Operações Assíncronas:** Utiliza ReturnsAsync e ThrowsAsync em IParceirosRepository para simular requisições assíncronas ao banco de dados sem I/O real.
- **Contexto de Sessão e Flash Data:** Configura um TempDataDictionary vinculado a um DefaultHttpContext mockado para asserções sobre mensagens de feedback (TempData["Sucesso"], TempData["Erro"] e ViewBag.Aviso).

Casos de Teste Mapeados

- **Index_DeveRetornarViewComListaPaginada**
 - **Ação Avaliada:** await _controller.Index(1, "busca", ...)
 - **Status/Retorno:** ViewResult com List<Parceiro>
 - **Propósito:** Garante a listagem filtrada e a popularização das metainformações de paginação na ViewBag.
- **Index_QuandoOcorrerExcecao_DeveRedirecionarParaError**
 - **Ação Avaliada:** await _controller.Index()
 - **Status/Retorno:** RedirectToActionResult (Home/Error)
 - **Propósito:** Trata falhas na busca e redireciona para a tela global de erro.
- **Details_ComIdVazio_DeveRetornarBadRequest**
 - **Ação Avaliada:** await _controller.Details("")
 - **Status/Retorno:** 400 Bad Request
 - **Propósito:** Bloqueia a consulta quando nenhum identificador é informado.
- **Details_QuandoNaoEncontrado_DeveRetornarNotFound**
 - **Ação Avaliada:** await _controller.Details("123")
 - **Status/Retorno:** 404 Not Found
 - **Propósito:** Responde adequadamente ao buscar um parceiro inexistente no banco.
- **Details_ComIdValido_DeveRetornarViewComParceiro**
 - **Ação Avaliada:** await _controller.Details("123")
 - **Status/Retorno:** ViewResult com Parceiro
 - **Propósito:** Valida a exibição dos detalhes de um parceiro válido.
- **Create_Get_DeveRetornarViewComNovoParceiroAtivo**
 - **Ação Avaliada:** _controller.Create()
 - **Status/Retorno:** ViewResult com Parceiro
 - **Propósito:** Garante que novos formulários venham com a flag Ativo = true por padrão.
- **Create_Post_ComModelStateInvalido_DeveRetornarView**
 - **Ação Avaliada:** await _controller.Create(parceiro)
 - **Status/Retorno:** ViewResult com o modelo
 - **Propósito:** Retorna o formulário preenchido sem chamar o repositório quando há erros de validação.
- **Create_Post_ComSucesso_DeveAdicionarERedirecionar**
 - **Ação Avaliada:** await _controller.Create(parceiro)
 - **Status/Retorno:** RedirectToActionResult("Index")
 - **Propósito:** Confirma o cadastro do parceiro e a mensagem de confirmação no TempData.
- **Edit_Post_SemAlteracoes_DeveDefinirViewBagAviso**
 - **Ação Avaliada:** await _controller.Edit(modeloFormulario)

- **Status/Retorno:** ViewResult + ViewBag.Aviso
- **Propósito:** Detecta equivalência de dados (desconsiderando máscaras) e cancela a atualização no banco.
- **Edit_Post_ComAlteracoes_DeveAtualizarERedirecionar**
 - **Ação Avaliada:** await _controller.Edit(modeloFormulario)
 - **Status/Retorno:** RedirectToActionResult("Index")
 - **Propósito:** Atualiza o registro garantindo que campos como Data_Cadastro fiquem intactos.
- **DeleteConfirmed_ComSucesso_DeveExcluir**
 - **Ação Avaliada:** await _controller.DeleteConfirmed("123")
 - **Status/Retorno:** RedirectToActionResult("Index")
 - **Propósito:** Valida a remoção física do registro no repositório.
- **AlternarStatus_AtivoTrue_DeveInativar**
 - **Ação Avaliada:** await _controller.AlternarStatus("123", true)
 - **Status/Retorno:** RedirectToActionResult("Index")
 - **Propósito:** Inverte a flag do parceiro para inativo (false).
- **AlternarStatus_AtivoFalse_DeveAtivar**
 - **Ação Avaliada:** await _controller.AlternarStatus("123", false)
 - **Status/Retorno:** RedirectToActionResult("Index")
 - **Propósito:** Inverte a flag do parceiro para ativo (true).

Detalhamento Técnico e Justificativas do Código

- **Edit_Post_SemAlteracoes_DeveDefinirViewBagAvisoERetornarView**

Telefone = "(11) 99999-9999" // Banco

Telefone = "11999999999" // Formulário

Assert.Equal("Nenhuma alteração foi realizada.", _controller.ViewBag.Aviso);

_mockRepo.Verify(r => r.Atualizar(It.IsAny<Parceiro>()), Times.Never);

- **Justificativa:** O teste garante que diferenças de formatação ou máscara no telefone não acionem atualizações no banco de dados (Times.Never).

- **Edit_Post_ComAlteracoes_DeveAtualizarERedirecionar**

var existente = new Parceiro { Id = "1", Nome = "Nome Antigo", Data_Cadastro = DateTime.UtcNow.AddDays(-10) };

await _controller.Edit(modeloFormulario);

Assert.Equal(existente.Data_Cadastro, modeloFormulario.Data_Cadastro);

_mockRepo.Verify(r => r.Atualizar(modeloFormulario), Times.Once);

- **Justificativa:** O formulário de edição não reenvia a data de criação original. O teste valida se a controller recupera o registro do banco e reatribui a Data_Cadastro original antes de persistir as mudanças.

- **AlternarStatus_AtivoTrue_DeveInativarEInformarSucesso**

var result = await _controller.AlternarStatus("123", true);

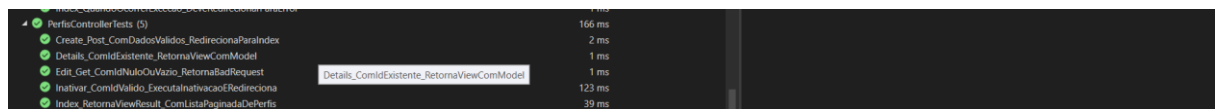
Assert.Equal("Parceiro inativado com sucesso.", _controller.TempData["Sucesso"]);

```
_mockRepo.Verify(r => r.AtualizarStatus("123", false), Times.Once);
```

- **Justificativa:** A ação do controller inverte o estado atual recebido na requisição. O teste valida que passar o estado true resulta em uma chamada com o novo valor false no repositório.

PerfisControllerTests

A classe `PerfisControllerTests` garante a integridade da gestão de controle de acesso baseado em funções (RBAC), validando a atribuição de permissões granulares, listagem paginada e inativação de perfis.



PerfisControllerTests (5)	166 ms
Create_Post_ComDadosValidos_RedirecionaParaIndex	2 ms
Details_ComIdExistente_RetornaViewComModel	1 ms
Edit_Get_ComIdNuloOuVazio_RetornaBadRequest	1 ms
Inativar_ComIdValido_ExecutaInativacaoERedireciona	123 ms
Index_RetornaViewResult_ComListaPaginadaDePerfis	39 ms

Estratégia de Configuração e Mocks

- **Mocking de Dependências de Segurança:** Utiliza a interface `IPerfisRepository` para simular a associação entre perfis de usuário e suas permissões sem chamadas ao banco.
- **Suporte à Sessão de Controller:** Inicializa o `TempDataDictionary` no construtor acoplado a um `DefaultHttpContext` para validar os fluxos de redirecionamento.

Casos de Teste Mapeados

- **Index_RetornaViewResult_ComListaPaginadaDePerfis**
 - **Ação Avaliada:** `await _controller.Index(1, null, null)`
 - **Status/Retorno:** `ActionResult` com `List<Perfil>`
 - **Propósito:** Assegura o retorno correto da listagem de perfis ativos para a view principal.
- **Create_Post_ComDadosValidos_RedirecionaParaIndex**
 - **Ação Avaliada:** `await _controller.Create(novoPerfil, permissoes)`
 - **Status/Retorno:** `RedirectToActionResult (Index)`
 - **Propósito:** Valida a criação do perfil vinculando a matriz de permissões associada.
- **Edit_Get_ComIdNuloOuVazio_RetornaBadRequest**
 - **Ação Avaliada:** `await _controller.Edit(null)`
 - **Status/Retorno:** `400 Bad Request`
 - **Propósito:** Intercepta tentativas de edição sem passar o parâmetro de identificação.
- **Details_ComIdExistente_RetornaViewComModel**
 - **Ação Avaliada:** `await _controller.Details("p123")`
 - **Status/Retorno:** `ActionResult` com `Perfil`
 - **Propósito:** Valida a recuperação e exibição do perfil correspondente ao ID informado.
- **Inativar_ComIdValido_ExecutaInativacaoERedireciona**
 - **Ação Avaliada:** `await _controller.Inativar("p123")`

- **Status/Retorno:** RedirectToActionResult (Index)
- **Propósito:** Garante a execução da regra de exclusão lógica/inativação no repositório.

Detalhamento Técnico e Justificativas do Código

• Create_Post_ComDadosValidos_RedirecionaParaIndex

```
var novoPerfil = new Perfil { Nome = "Gerente", Ativo = true };
string[] permissoes = new[] { "Perfis.Gerenciar" };
```

```
var result = await _controller.Create(novoPerfil, permissoes);
```

```
_repositoryMock.Verify(r => r.Adicionar(It.IsAny<Perfil>()), Times.Once);
```

- **Justificativa:** O cadastro recebe o modelo e um array suplementar de permissões (string[] permissoes). O teste valida se a inclusão é processada e aciona o método Adicionar no repositório uma única vez.

• Inativar_ComIdValido_ExecutaInativacaoERedireciona

```
_repositoryMock.Setup(r => r.Inativar("p123")).Returns(Task.CompletedTask);
```

```
var result = await _controller.Inativar("p123");
```

```
var redirectResult = Assert.IsType<RedirectToActionResult>(result);
```

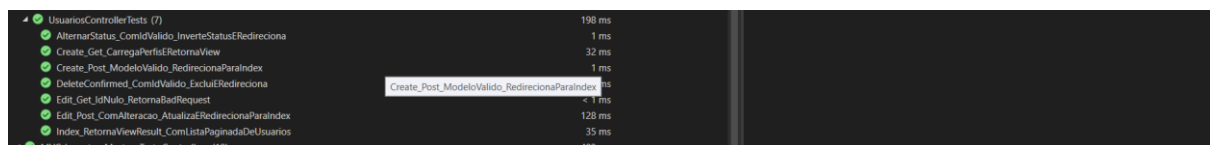
```
Assert.Equal("Index", redirectResult.ActionName);
```

```
_repositoryMock.Verify(r => r.Inativar("p123"), Times.Once);
```

- **Justificativa:** Perfis de acesso não são apagados fisicamente para manter a rastreabilidade histórica. O teste confirma a execução de Inativar("p123") e o redirecionamento para a listagem.

UsuariosControllerTests

A classe UsuariosControllerTests valida as operações de gerenciamento de usuários do sistema, cobrindo criação com seleção de perfis (ViewBag.Perfis), atualização de dados, exclusão e alteração de status.



UsuariosControllerTests (7)	198 ms
AlterarStatus_ComIdValido_InverteStatusERedireciona	1 ms
Create_Get_CarregaPerfisERetornaView	32 ms
Create_Post_ModeloValido_RedirecionaParaIndex	1 ms
DeleteConfirmed_ComIdValido_ExcluiERedireciona	1 ms
Edit_Get_IdNulo_RetornaBadRequest	< 1 ms
Edit_Post_ComAlteracao_AtualizaERedirecionaParaIndex	128 ms
Index_RetornaViewResult_ComListaPaginadaDeUsuarios	35 ms
UsuariosControllerTests (7)	493 ms

Estratégia de Configuração e Mocks

- **Isolamento de Repositórios Múltiplos:** Injeta mocks de IUserariosRepository e IPerfisRepository para testar o relacionamento entre dados de usuários e a montagem do seletor de perfil na interface.
- **Contexto de Requisição Mockado:** Inicializa o TempDataDictionary vinculado a um DefaultHttpContext para garantir a validação de redirecionamentos sem erros.

Casos de Teste Mapeados]

- **Index_RetornaViewResult_ComListaPaginadaDeUsuarios**
 - **Ação Avaliada:** await _controller.Index(1, ...)
 - **Status/Retorno:** ViewResult com List<Usuario>
 - **Propósito:** Confirma a recuperação e renderização da listagem paginada de usuários.
- **Create_Get_CarregaPerfisERetornaView**
 - **Ação Avaliada:** await _controller.Create()
 - **Status/Retorno:** ViewResult + ViewBag.Perfis (List<SelectListItem>)
 - **Propósito:** Valida o pré-carregamento dos perfis de acesso na ViewBag para popular o elemento <select> do formulário.
- **Create_Post_ModeloValido_RedirecionaParaIndex**
 - **Ação Avaliada:** await _controller.Create(novoUsuario)
 - **Status/Retorno:** RedirectToActionResult (Index)
 - **Propósito:** Assegura a inclusão do novo usuário e redirecionamento para a tela principal.
- **Edit_Get_IdNulo_RetornaBadRequest**
 - **Ação Avaliada:** await _controller.Edit((string?)null)
 - **Status/Retorno:** 400 Bad Request
 - **Propósito:** Bloqueia a requisição de edição caso o parâmetro identificador seja nulo.
- **Edit_Post_ComAlteracao_AtualizaERedirecionaParaIndex**
 - **Ação Avaliada:** await _controller.Edit(usuarioEditado)
 - **Status/Retorno:** RedirectToActionResult (Index)
 - **Propósito:** Assegura a persistência das alterações no repositório e redirecionamento da página.
- **DeleteConfirmed_ComIdValido_ExcluiERedireciona**
 - **Ação Avaliada:** await _controller.DeleteConfirmed("u123")
 - **Status/Retorno:** RedirectToActionResult (Index)
 - **Propósito:** Valida a execução da remoção do usuário pelo ID correspondente.
- **AlternarStatus_ComIdValido_InverteStatusERedireciona**
 - **Ação Avaliada:** await _controller.AlternarStatus("u123", true)
 - **Status/Retorno:** RedirectToActionResult (Index)
 - **Propósito:** Transiciona a flag Ativo do usuário para o valor oposto (false).

Detalhamento Técnico e Justificativas do Código

- **Create_Get_CarregaPerfisERetornaView**

```
var perfisViewBag =
```

```
Assert.IsAssignableFrom<List<SelectListItem>>(_controller.ViewBag.Perfis);
```

```
Assert.Single(perfisViewBag);
```

```
Assert.Equal("Administrador", perfisViewBag[0].Value);
```

- **Justificativa:** O cadastro depende dos perfis cadastrados para popular o dropdown. O teste valida se a controller busca os dados em IPerfisRepository e os projeta em objetos SelectListItem dentro da ViewBag.

- **Edit_Post_ComAlteracao_AtualizaERedirecionaParaIndex**

```
_usuariosRepositoryMock.Setup(r => r.BuscarPorId("u1")).ReturnsAsync(usuarioOriginal);
```

```
var result = await _controller.Edit(usuarioEditado);
```

```
_usuariosRepositoryMock.Verify(r => r.Atualizar(It.IsAny<Usuario>()), Times.Once);
```

- **Justificativa:** Para evitar que campos não editáveis no formulário (como Senha ou EmpresaId) sejam sobrescritos por nulo, a controller busca o registro original no banco antes de aplicar as mudanças e atualizar.
- **AlternarStatus_ComIdValido_InverteStatusERedireciona**

```
var result = await _controller.AlternarStatus("u123", true);
```

```
_usuariosRepositoryMock.Verify(r => r.AtualizarStatus("u123", false), Times.Once);
```

- **Justificativa:** Em vez de remover usuários e afetar a integridade referencial, a interface alterna o status. O teste valida se ao passar o estado atual true (ativo), o parâmetro invertido false (inativo) é repassado ao repositório.